函数返回指针的四条死路：三条炸，一条最隐蔽

嵌入式 Linux 手记　2026-10-17　公众号文章 · 原链接 mp.weixin.qq.com

```c
char *a(void) {
    char buf[32] = "benjamin";
    return buf;         /* gcc 默认就警告：address of local variable returned */
}

char *b(void) {
    staticchar out[32] = "benjamin";
    return out;         /* 不报警告，能跑，但埋伏在后面 */
}

char *c(void) {
    char *p = malloc(32);
    memcpy(p, "benjamin", 9);
    return p;           /* 也不报警告，埋伏换了个地方 */
}
```

三个函数都编译通过，前两个甚至只差一个 static。返回指针的路上，编译器的警告只够吓退第一条；后面三条，一条比一条安静。上一篇讲过，free 之后的堆内存是块没人认领的地，悬空指针读旧值是赛跑结果。这一次进函数内部，看指针从 return 交出来的四条死路与两条活路。

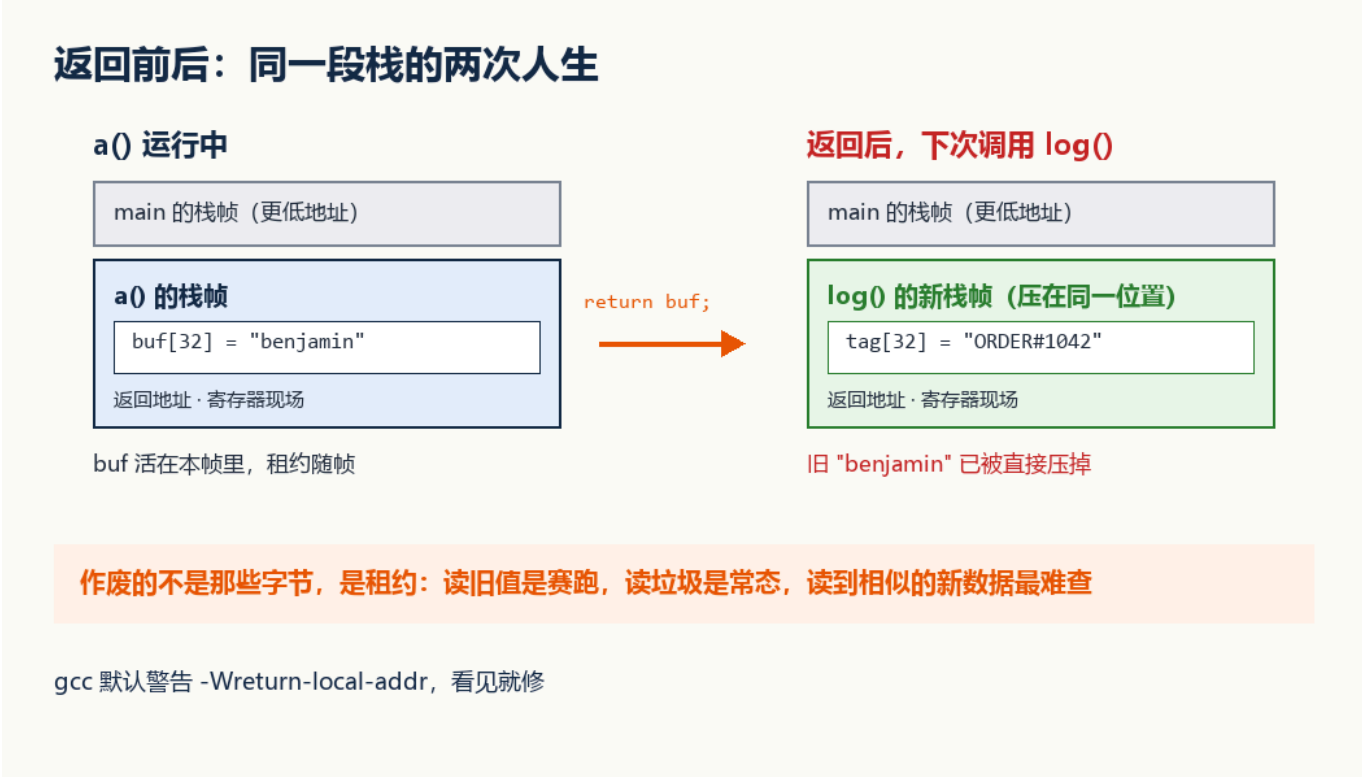
一、死路一：栈上的地址，出函数就作废

`a()` 返回的是栈上数组 `buf` 的地址。局部变量活在它的栈帧里，函数返回，栈帧弹出，那 32 个字节的"官方租约"就到期了。

作废之后去哪儿了？物理上哪儿也没去。字节还在原地，只是这块栈空间已经被宣告自由：下一次函数调用会在同一位置重建栈帧，新帧的局部变量直接压在旧地址上。

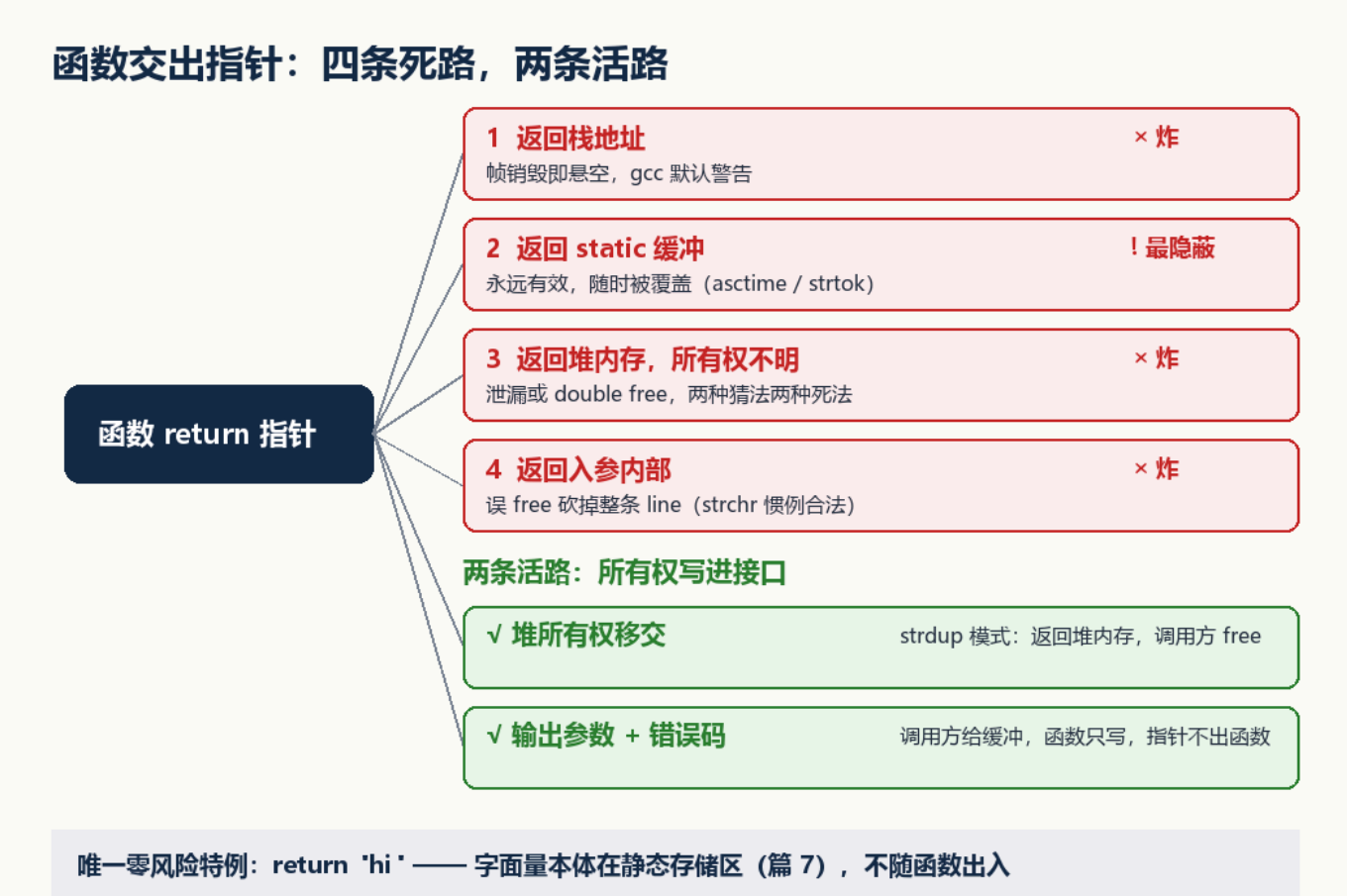
所以 `a()` 的返回值有时还能读出 "benjamin"——又是赛跑结果，和篇 8 的悬空堆内存一个道理，只是这里的"下一个住客"来得格外快：你几乎做任何事（打个 printf、调个库函数）都会踩过那块栈。

读出旧值算运气，读出垃圾算正常，最难受的是第三种：新住客写进了结构相似的数据，程序不崩、逻辑看似通，只是某些值偶尔不对。这类 bug 的案发现场，可能离案发时间隔了几百万次调用，栈上不留任何痕迹。



图：函数返回后栈帧弹出，下次调用在同一位置重建，旧数据被直接覆盖

顺带校准一个流行误区：返回 `return "hi"` 不是这条死路。字面量本体在静态存储区（篇 7 拆过，C11 §6.4.5），不随栈帧出入，返回后一直有效。返回 static 缓冲虽也永远有效，但内容可能被下一次调用覆盖；`return "hi"` 则连内容都稳如泰山，是唯一无需附加条件的零风险返回，靠的不是运气，是存储位置。



图：四条死路与两条活路：三条踩了就炸，一条最隐蔽，活路全靠写清所有权

二、死路二：static 不是解药，是更深的坑

很多人被栈吓过之后学"聪明"了：加个 static，变量进入静态存储区，生命周期贯穿整个程序，返回值永远有效。`b()` 就是这么写的，而且它确实"永远有效"。

问题出在"共享"两个字上。static 缓冲全程序只有一份，每一次调用都写同一块地：

```
char *upper(const char *s) {
    static char out[64];
    size_t i = 0;
    for (; s[i] && i < 63; i++)
        out[i] = (char)(s[i] >= 'a' && s[i] <= 'z' ? s[i] - 32 : s[i]);
    out[i] = '\0';
    return out;
}

printf("%s %s\n", upper("hello"), upper("world"));
/* 打印 WORLD WORLD 或 HELLO HELLO，两个一样，反正不是你要的 */
```

单看每次调用都对，组合起来全错。实参求值顺序标准未规定（C11 §6.5.2.2），但无所谓：无论谁先跑，两次调用写的是同一块 static，printf 真正打印时，缓冲里只剩后跑那次的 "WORLD"（或 "HELLO"）。单线程都埋着这种雷，多线程更不用说，两个线程同时写一块无锁缓冲，比覆盖更疯的输出都可能发生。

这不是自研代码才有的坑，标准库里有现成教材。`asctime` 返回的字符串躺在静态缓冲里，下一次调用 `asctime` 或 `ctime`，上一次的结果就被覆盖（POSIX 明文允许）；`strtok` 更进一步，把解析进度存在静态存储里，两处代码交替解析就会互相搅乱位置，所以它线程不安全。glibc 提供 `strtok_r`、C11 附录 K 提供 `strtok_s`，让调用方自带进度指针，等于官方承认了这个设计的代价。



图：两次调用写同一块 static，先到者的结果在第二次调用时消失

自研接口的对应规约：返回静态缓冲的函数，文档必须写明"下次调用失效"；调用方要长期持有，拿到手就立刻拷贝。

三、死路三与死路四：所有权不明，指向入参内部

`c()` 返回堆内存，不炸也不覆盖，看起来最体面。它的埋伏在接口之外：调用方拿到指针，第一件事该问"这块内存归我管吗？我要 free 吗？"没人回答。猜着不 free 的，内存泄漏，程序在长跑第 N 天被 OOM 杀掉；猜着 free 的，而这块内存还被第二个地方引用着，free 完别处就是悬空指针，double free 当场崩。同一个函数，两种猜法，两种死法。这是"踩了就炸"的第三条。

死路四更绕：返回的不是新内存，是入参的内部。

```c
char *field_after(char *line) {
    char *comma = strchr(line, ',');
    return comma ? comma + 1 : NULL;   /* 返回值是 line 的一段 */
}
```

返回值指向 `line` 中间某处。调用方按"函数返回堆内存"的惯性对它 `free`，砍掉的却是整条 `line`：`line` 悬空，之前从它切出来的所有"字段指针"集体遭殃。一次 free，多个受害者。

分寸要钉准：返回入参内部指针本身不是错，`strchr` / `strstr` 就是这么干的，`string.h` 的惯例一贯是"不分配、不转移所有权"。错在接口没说清"返回值指向谁的内存、要不要 free"。`strtok` 是两宗罪集一身的教材：解析进度藏进静态区（死路二），返回值又指向你的字符串内部（死路四），所以它才值得官方专门补救。

四、两条活路：把所有权写进接口

活路一，堆所有权移交。strdup 模式：返回堆内存，文档写明"返回值由调用方 free"，所有权随指针一起过户：

```c
/* 返回堆内存，调用方负责 free（C23 已将 strdup 收编进标准库） */
char *dup_name(const char *s) {
    size_t n = strlen(s) + 1;
    char *p = malloc(n);
    if (p == NULL) return NULL;    /* 失败路径也要有约定 */
    memcpy(p, s, n);
    return p;
}
```

活路二，输出参数。调用方自己提供缓冲区，函数只写不分配，指针根本不出函数；返回值改报错误码，Linux 内核的接口大多长这样：

```c
/* 调用方给缓冲，函数只写；返回 0 成功，负数错误码 */
int get_name(char *out, size_t cap) {
    if (out == NULL || cap < 9) return -1;
    memcpy(out, "benjamin", 9);    /* 8 字符 + '\0'，长度跟着缓冲走 */
    return 0;
}
```

标准库的现成范本是 `strftime`：缓冲、容量都由调用方给，函数想写也写不越界。

两条活路怎么选？小而固定的数据（时间戳、状态码字符串）用输出参数，栈上就地解决；大小运行期才知道、或者需要跨层传递的数据，用堆所有权移交。共同点只有一个：分配和释放的责任，必须有一条清晰的、写在注释里的主线。

写在最后

1. 返回栈地址是死路：帧销毁即悬空，读旧值是赛跑不是承诺；gcc 默认警告，看见就修。
2. static 返回值"永远有效"但"随时被覆盖"：连续调用互相踩、多线程互相抢；要长期持有，拿到就拷贝。`asctime`/`strtok` 是标准库的现成教材。
3. 返回堆内存必须写清所有权：谁分配谁释放，注释即契约；`strdup` 模式（C23 已收编）是最简洁的移交，失败返回 NULL 也要写进约定。
4. 返回入参内部指针时声明"不要 free"（`strchr` 惯例合法，误会才是坑）；拿不准就改用输出参数 + 错误码：调用方给缓冲，函数只写，指针不出函数。
5. `return "hi"` 永远合法：字面量在静态存储区，与栈帧无关——这是唯一无需附加条件的零风险返回指针。

死路都绕开了，还有一类代码把"改指针本身"写得战战兢兢：删个链表头节点，都要 `if` 特判"它是不是头"。Linus 眼里这是二流写法。下一篇《删除链表头节点，Linus 只用了一个二级指针》，看他怎么用一个间接指针消掉全部特例。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。